SQL Code with Internal Documentation

```sql
/* ============================================================
  PHASE 0 – DATA EXPLORATION: UNDERSTAND SOURCE TABLES
  ============================================================ */

-- Quick structure & sample check of base tables
SELECT TOP 100 * FROM Sales.SalesOrderHeader;
SELECT TOP 100 * FROM Sales.SalesOrderDetail;
SELECT TOP 100 * FROM Production.Product;
SELECT TOP 100 * FROM Production.ProductSubcategory;
SELECT TOP 100 * FROM Production.ProductCategory;

-- Basic join sanity check: do the tables connect as expected?
SELECT TOP 100 *
FROM Sales.SalesOrderHeader              AS h
INNER JOIN Sales.SalesOrderDetail         AS d    ON h.SalesOrderID = d.SalesOrderID
INNER JOIN Production.Product             AS p    ON d.ProductID    = p.ProductID
LEFT JOIN Production.ProductSubcategory    AS ps   ON p.ProductSubcategoryID = ps.ProductSubcategoryID
LEFT JOIN Production.ProductCategory       AS pc   ON ps.ProductCategoryID   = pc.ProductCategoryID;

/* ============================================================
  PHASE 1 – CREATE ANALYSIS PANEL (General_Panel)
  ------------------------------------------------------------
  With this one Panel, you can calculate:
  - seasonality (group by month/quarter)
  - trends (group by year, month)
  - discount vs margin (group by discount buckets)
  - category profitability (CategoryName / SubcategoryName)
  - quantity vs margin (OrderQty, etc.)
  ============================================================ */

-- Drop existing view if it exists (for re-runs)
IF OBJECT_ID('dbo.General_Panel', 'V') IS NOT NULL
   DROP VIEW dbo.General_Panel;
GO

CREATE VIEW dbo.General_Panel
AS
SELECT
   /* Keys & time */
   h.SalesOrderID,
   d.SalesOrderDetailID,
   h.OrderDate,
   YEAR(h.OrderDate)            AS OrderYear,
   MONTH(h.OrderDate)           AS OrderMonthNumber,
   DATENAME(month, h.OrderDate)   AS OrderMonthName,
   DATEPART(QUARTER, h.OrderDate) AS OrderQuarter,

   /* Product & category */
   d.ProductID,
   p.Name                      AS ProductName,
   ps.Name                     AS SubcategoryName,
   pc.Name                     AS CategoryName,

   /* Quantities & prices */
   d.OrderQty,
   d.UnitPrice,
   d.UnitPriceDiscount          AS DiscountPercent,
   d.UnitPrice * (1 - d.UnitPriceDiscount) AS NetUnitPrice,

   /* Financials */
   d.LineTotal                 AS LineRevenue,      -- sales revenue
   p.StandardCost,
   p.StandardCost * d.OrderQty    AS LineCost,         -- cost of goods
   (d.UnitPrice - p.StandardCost) AS UnitMargin,       -- unit-level margin
   d.LineTotal - (d.OrderQty * p.StandardCost) AS LineProfit,
   (d.LineTotal - (d.OrderQty * p.StandardCost))
      / NULLIF(d.LineTotal, 0)   AS MarginPercent

FROM Sales.SalesOrderHeader           AS h
INNER JOIN Sales.SalesOrderDetail      AS d
      ON h.SalesOrderID = d.SalesOrderID
INNER JOIN Production.Product          AS p
      ON d.ProductID    = p.ProductID
LEFT JOIN Production.ProductSubcategory AS ps
      ON p.ProductSubcategoryID = ps.ProductSubcategoryID
LEFT JOIN Production.ProductCategory    AS pc
      ON ps.ProductCategoryID   = pc.ProductCategoryID;
GO

-- Quick panel check
SELECT TOP 100 * FROM dbo.General_Panel;

/* ============================================================
  PHASE 2 – DATA QUALITY / INTEGRITY CHECKS
  ============================================================ */

-- 2.1 Basic shape of the dataset
SELECT
   COUNT(*)                  AS TotalRows,
   MIN(OrderDate)             AS MinOrderDate,
   MAX(OrderDate)             AS MaxOrderDate,
   COUNT(DISTINCT SalesOrderID) AS DistinctOrders,
   COUNT(DISTINCT ProductID)    AS DistinctProducts
FROM General_Panel;

-- 2.2 NULL checks in key fields
SELECT
   SUM(CASE WHEN OrderDate IS NULL       THEN 1 ELSE 0 END) AS NullOrderDate,
   SUM(CASE WHEN OrderYear IS NULL       THEN 1 ELSE 0 END) AS NullOrderYear,
   SUM(CASE WHEN OrderMonthNumber IS NULL THEN 1 ELSE 0 END) AS NullOrderMonthNumber,
   SUM(CASE WHEN OrderQty IS NULL        THEN 1 ELSE 0 END) AS NullOrderQty,
   SUM(CASE WHEN UnitPrice IS NULL       THEN 1 ELSE 0 END) AS NullUnitPrice,
   SUM(CASE WHEN StandardCost IS NULL    THEN 1 ELSE 0 END) AS NullStandardCost,
   SUM(CASE WHEN LineRevenue IS NULL     THEN 1 ELSE 0 END) AS NullLineRevenue
FROM General_Panel;
-- Result: N/A (no NULLs in key fields)

-- 2.3 Check for weird or impossible values
SELECT
   SUM(CASE WHEN OrderQty <= 0     THEN 1 ELSE 0 END) AS NonPositiveQty,
   SUM(CASE WHEN UnitPrice <= 0   THEN 1 ELSE 0 END) AS NonPositivePrice,
   SUM(CASE WHEN StandardCost < 0 THEN 1 ELSE 0 END) AS NegativeCost,
   SUM(CASE WHEN LineRevenue <= 0 THEN 1 ELSE 0 END) AS NonPositiveRevenue,
   SUM(CASE WHEN LineProfit < 0   THEN 1 ELSE 0 END) AS NegativeProfitRows
FROM General_Panel;
-- 29,161 lines with negative profit (important for later analysis)

-- 2.4 Category-related fields completeness
SELECT
   SUM(CASE WHEN SubcategoryName IS NULL THEN 1 ELSE 0 END) AS NoSubcategory,
   SUM(CASE WHEN CategoryName    IS NULL THEN 1 ELSE 0 END) AS NoCategory
FROM General_Panel;
-- Result: N/A (no missing category info in panel)

-- 2.5 Discount sanity
SELECT
   MIN(DiscountPercent) AS MinDiscount,
   MAX(DiscountPercent) AS MaxDiscount,
   AVG(DiscountPercent) AS AvgDiscount
FROM General_Panel;
-- Detected discounts ~ 0–40%

-- 2.6 Discount distribution (buckets)
SELECT
   CASE
      WHEN DiscountPercent IS NULL      THEN 'NULL'
      WHEN DiscountPercent = 0          THEN '0%'
      WHEN DiscountPercent > 0
          AND DiscountPercent <= 0.10  THEN '0–10%'
      WHEN DiscountPercent > 0.10
          AND DiscountPercent <= 0.30  THEN '10–30%'
      WHEN DiscountPercent > 0.30       THEN '30%+'
   END AS DiscountBucket,
   COUNT(*) AS NoOfRows
FROM General_Panel
GROUP BY
   CASE
      WHEN DiscountPercent IS NULL      THEN 'NULL'
      WHEN DiscountPercent = 0          THEN '0%'
      WHEN DiscountPercent > 0
          AND DiscountPercent <= 0.10  THEN '0–10%'
      WHEN DiscountPercent > 0.10
          AND DiscountPercent <= 0.30  THEN '10–30%'
      WHEN DiscountPercent > 0.30       THEN '30%+'
   END
ORDER BY DiscountBucket;

-- 2.7 Sales order status – to detect cancelled orders
SELECT DISTINCT Status
FROM Sales.SalesOrderHeader;
-- There are no cancelled orders (no need to filter by status)

-- 2.8 OrderQty distribution
SELECT
   OrderQty,
   COUNT(*) AS NoOfTimes
FROM General_Panel
GROUP BY OrderQty
ORDER BY OrderQty;
-- Distribution looks reasonable (1–40+)

-- 2.9 UnitPrice feasibility
SELECT
   MAX(UnitPrice)    AS MaxUnitPrice,
   MIN(UnitPrice)    AS MinUnitPrice,
   SUM(UnitPrice)    AS SumUnitPrice,
   AVG(UnitPrice)    AS AvgUnitPrice,
   COUNT(UnitPrice)  AS LineCount
FROM General_Panel;
-- Values are logical

-- 2.10 Check for NULL / missing ProductIDs in panel
SELECT COUNT(*) AS NullProductIDCount
FROM General_Panel
WHERE ProductID IS NULL;
-- N/A

-- 2.11 Zero or negative StandardCost in source vs panel
SELECT COUNT(*) AS ZeroOrNegativeCostProducts
FROM Production.Product
WHERE StandardCost <= 0;
-- ~200 products in source (never show up in panel as cost=0)

SELECT COUNT(*) AS ZeroCostInPanel
FROM General_Panel
WHERE StandardCost = 0;
-- None in panel (we are safe for margin calculations)

-- 2.12 StandardCost distribution (to detect extremes)
SELECT TOP 10 StandardCost
FROM General_Panel
ORDER BY StandardCost DESC;

SELECT TOP 10 StandardCost
FROM General_Panel
ORDER BY StandardCost ASC;

-- 2.13 Min/max for revenue, cost, profit, margin%
SELECT
   MIN(LineRevenue)   AS MinRevenue,
   MAX(LineRevenue)   AS MaxRevenue,
   MIN(LineCost)      AS MinCost,
   MAX(LineCost)      AS MaxCost,
   MIN(LineProfit)    AS MinProfit,
   MAX(LineProfit)    AS MaxProfit,
   MIN(MarginPercent) AS MinMarginPercent,
   MAX(MarginPercent) AS MaxMarginPercent
FROM General_Panel;
-- Many negative profit values – key issue for analysis

-- 2.14 Check for duplicate (SalesOrderID, ProductID) lines
SELECT
   SalesOrderID,
   ProductID,
   COUNT(*) AS LineCount
FROM General_Panel
GROUP BY SalesOrderID, ProductID
HAVING COUNT(*) > 1;
-- There are no duplicates

-- 2.15 Margin sanity by category (at aggregated level)
SELECT
   CategoryName,
   COUNT(*)                AS NoOfRows,
   SUM(LineRevenue)        AS TotalRevenue,
   SUM(LineProfit)         AS TotalProfit,
   SUM(LineProfit) / NULLIF(SUM(LineRevenue), 0) AS MarginPercent
FROM General_Panel
GROUP BY CategoryName
ORDER BY TotalRevenue DESC;

/* ============================================================
  PHASE 3 – SEASONALITY & TREND ANALYSIS
  ============================================================ */

-- 3.1 Simple monthly seasonality (raw sums)
SELECT
   OrderMonthNumber,
   OrderMonthName,
   SUM(LineRevenue) AS TotalRevenue,
   SUM(LineProfit)  AS TotalProfit,
   SUM(LineProfit) / NULLIF(SUM(LineRevenue), 0) AS MarginPercent
FROM General_Panel
GROUP BY OrderMonthNumber, OrderMonthName
ORDER BY OrderMonthNumber;

-- 3.2 Simple quarterly seasonality (raw sums)
SELECT
   OrderQuarter,
   SUM(LineRevenue) AS TotalRevenue,
   SUM(LineProfit)  AS TotalProfit,
   SUM(LineProfit) / NULLIF(SUM(LineRevenue), 0) AS MarginPercent
FROM General_Panel
GROUP BY OrderQuarter
ORDER BY OrderQuarter;

-- 3.3 Year–month trends (for "heatmap" analysis in Excel)
SELECT
   OrderYear,
   OrderMonthNumber,
   OrderMonthName,
   SUM(LineRevenue) AS TotalRevenue,
   SUM(LineProfit)  AS TotalProfit,
   SUM(LineProfit) / NULLIF(SUM(LineRevenue), 0) AS MarginPercent
FROM General_Panel
GROUP BY OrderYear, OrderMonthNumber, OrderMonthName
ORDER BY OrderYear, OrderMonthNumber;

-- 3.4 Simple yearly trend (raw sums)
SELECT
   OrderYear,
   SUM(LineRevenue) AS TotalRevenue,
   SUM(LineProfit)  AS TotalProfit,
   SUM(LineProfit) / NULLIF(SUM(LineRevenue), 0) AS MarginPercent
FROM General_Panel
GROUP BY OrderYear
ORDER BY OrderYear;

-- 3.5 Number of months available per year (partial years)
SELECT
   OrderYear,
   COUNT(DISTINCT OrderMonthNumber) AS MonthsAvailable
FROM General_Panel
GROUP BY OrderYear
ORDER BY OrderYear;
/*
2011 and 2014 are partial years in the dataset.
Trend conclusions should rely primarily on normalized monthly averages
and full years (2012–2013) where needed.
*/

-- 3.6 Yearly trend with normalized (per-month) values
WITH YearSummary AS (
   SELECT
      OrderYear,
      SUM(LineRevenue) AS TotalRevenue,
      SUM(LineProfit)  AS TotalProfit,
      SUM(LineProfit) / NULLIF(SUM(LineRevenue), 0) AS MarginPercent,
      COUNT(DISTINCT OrderMonthNumber) AS MonthsAvailable
   FROM General_Panel
   GROUP BY OrderYear
)
SELECT
   OrderYear,
   TotalRevenue,
   TotalProfit,
   MarginPercent,
   MonthsAvailable,
   TotalRevenue / NULLIF(MonthsAvailable, 0) AS AvgMonthlyRevenue,
   TotalProfit  / NULLIF(MonthsAvailable, 0) AS AvgMonthlyProfit
FROM YearSummary
ORDER BY OrderYear;

-- 3.7 Corrected monthly seasonality (normalized by years with data)
SELECT
   OrderMonthNumber,
   MAX(OrderMonthName) AS OrderMonthName,
   COUNT(DISTINCT OrderYear) AS YearsWithData,
   SUM(LineRevenue)        AS TotalRevenueAllYears,
   SUM(LineProfit)         AS TotalProfitAllYears,
   SUM(LineRevenue) / NULLIF(COUNT(DISTINCT OrderYear), 0) AS AvgMonthlyRevenue,
   SUM(LineProfit)  / NULLIF(COUNT(DISTINCT OrderYear), 0) AS AvgMonthlyProfit,
   SUM(LineProfit) / NULLIF(SUM(LineRevenue), 0)          AS MarginPercent
FROM General_Panel
GROUP BY OrderMonthNumber
ORDER BY OrderMonthNumber;

-- 3.8 Corrected quarterly seasonality (normalized by years with data)
SELECT
   OrderQuarter,
   COUNT(DISTINCT OrderYear) AS YearsWithData,
   SUM(LineRevenue)         AS TotalRevenueAllYears,
   SUM(LineProfit)          AS TotalProfitAllYears,
   SUM(LineRevenue) / NULLIF(COUNT(DISTINCT OrderYear), 0) AS AvgQuarterlyRevenue,
   SUM(LineProfit)  / NULLIF(COUNT(DISTINCT OrderYear), 0) AS AvgQuarterlyProfit,
   SUM(LineProfit) / NULLIF(SUM(LineRevenue), 0)          AS MarginPercent
FROM General_Panel
GROUP BY OrderQuarter
ORDER BY OrderQuarter;

/* ============================================================
  PHASE 4 – PROFITABILITY DRIVERS
  4.1 Quantity vs Margin
  4.2 Discount vs Margin
  4.3 Category & Product mix
  ============================================================ */

-------------------------
-- 4.1 QUANTITY vs MARGIN
-------------------------

-- Transaction-level scatter data (for Excel scatter plot)
SELECT
   OrderQty,
   MarginPercent
FROM General_Panel
WHERE MarginPercent IS NOT NULL
ORDER BY OrderQty;          -- (and optionally DESC for second look)

-- Group by quantity: average margin per OrderQty
SELECT
   OrderQty,
   AVG(MarginPercent) AS AvgMarginPercent
FROM General_Panel
WHERE MarginPercent IS NOT NULL
GROUP BY OrderQty
ORDER BY OrderQty;

-- Monthly averages for interpretation: quantity & margin by month
SELECT
   OrderYear,
   OrderMonthNumber,
   OrderMonthName,
   AVG(OrderQty)     AS AvgQty,
   AVG(MarginPercent) AS AvgMargin,
   SUM(OrderQty)     AS TotalSoldQty
FROM General_Panel
GROUP BY
   OrderYear,
   OrderMonthNumber,
   OrderMonthName
ORDER BY
   OrderYear,
   OrderMonthNumber;
-- Interpretation: profitability is not driven directly by quantity,
-- but by product mix & discounts (quantity is a symptom, not root cause).

-------------------------
-- 4.2 DISCOUNT vs MARGIN
-------------------------

-- Transaction-level scatter data
SELECT
   DiscountPercent,
   MarginPercent
FROM General_Panel
WHERE MarginPercent IS NOT NULL
ORDER BY DiscountPercent;

-- Discount buckets: impact on margin & profit
SELECT
   CASE
      WHEN DiscountPercent = 0         THEN '0%'
      WHEN DiscountPercent <= 0.10     THEN '0–10%'
      WHEN DiscountPercent <= 0.30     THEN '10–30%'
      ELSE '30%+'
   END AS DiscountBucket,
   AVG(MarginPercent) AS AvgMargin,
   SUM(LineRevenue)   AS Revenue,
   SUM(LineProfit)    AS Profit,
   COUNT(*)           AS Transactions
FROM General_Panel
GROUP BY
   CASE
      WHEN DiscountPercent = 0         THEN '0%'
      WHEN DiscountPercent <= 0.10     THEN '0–10%'
      WHEN DiscountPercent <= 0.30     THEN '10–30%'
      ELSE '30%+'
   END
ORDER BY DiscountBucket;
-- Insight: higher discounts strongly reduce margin and
-- can push many orders into negative profitability.

-- Discount vs margin by month (for trend view)
SELECT
   OrderYear,
   OrderMonthNumber,
   OrderMonthName,
   AVG(DiscountPercent) AS AvgDiscount,
   AVG(MarginPercent)   AS AvgMargin
FROM General_Panel
GROUP BY
   OrderYear,
   OrderMonthNumber,
   OrderMonthName
ORDER BY
   OrderYear,
   OrderMonthNumber;

-- Investigating April 2012 (strongly negative month)
SELECT *
FROM General_Panel
WHERE OrderYear = 2012
  AND OrderMonthNumber = 4;

-- Same month – only rows with discount
SELECT *
FROM General_Panel
WHERE OrderYear = 2012
  AND OrderMonthNumber = 4
  AND DiscountPercent > 0;
-- Almost all discounted lines have negative profit and margin.

-- Distinct discounts overall
SELECT DISTINCT DiscountPercent
FROM General_Panel
ORDER BY DiscountPercent;

-- Distinct discounts in April 2012
SELECT DISTINCT DiscountPercent
FROM General_Panel
WHERE OrderYear = 2012
  AND OrderMonthNumber = 4
ORDER BY DiscountPercent;

-- Are there negative profit lines even with no discount?
SELECT *
FROM General_Panel
WHERE OrderYear = 2012
  AND OrderMonthNumber = 4
  AND DiscountPercent = 0
  AND LineProfit <= 0;
-- 557 lines have negative values despite no discounts
-- → root cause is product mix and cost structure (bikes).

-- See how widespread this "no-discount but negative profit" problem is
SELECT
   OrderYear,
   OrderMonthNumber,
   COUNT(*) AS NegativeLines
FROM General_Panel
WHERE DiscountPercent = 0
  AND LineProfit <= 0
GROUP BY OrderYear, OrderMonthNumber
ORDER BY OrderYear, OrderMonthNumber;
-- Many months show negative-profit lines even at 0% discount:
-- especially in weak-margin months.

--------------------------------------
-- 4.3 CATEGORY & PRODUCT PROFITABILITY
--------------------------------------

-- Actual profit by Category/Subcategory from panel (based on real sales)
SELECT
   CategoryName,
   SubcategoryName,
   AVG(LineProfit) AS AvgProfit,
   SUM(LineProfit) AS TotalProfit
FROM General_Panel
GROUP BY CategoryName, SubcategoryName
ORDER BY AvgProfit DESC;

-- Same, sorted by total profit contribution
SELECT
   CategoryName,
   SubcategoryName,
   AVG(LineProfit) AS AvgProfit,
   SUM(LineProfit) AS TotalProfit
FROM General_Panel
GROUP BY CategoryName, SubcategoryName
ORDER BY TotalProfit DESC;

-- Theoretical product profitability (ListPrice – StandardCost)
-- for products that HAVE been sold (only products present in panel)
SELECT
   pc.Name AS CategoryName,
   ps.Name AS SubcategoryName,
   AVG(p.ListPrice - p.StandardCost) AS AvgProfit,
   AVG((p.ListPrice - p.StandardCost) / NULLIF(p.ListPrice, 0)) AS AvgMarginPercent,
   COUNT(DISTINCT p.ProductID) AS ProductCount,
   SUM(gp.OrderQty) AS TotalOrderQty
FROM General_Panel AS gp
JOIN Production.Product AS p                 ON gp.ProductID = p.ProductID
LEFT JOIN Production.ProductSubcategory AS ps  ON p.ProductSubcategoryID = ps.ProductSubcategoryID
LEFT JOIN Production.ProductCategory    AS pc  ON ps.ProductCategoryID = pc.ProductCategoryID
GROUP BY pc.Name, ps.Name
ORDER BY AvgProfit DESC;

-- Same theoretical view, sorted by margin%
SELECT
   pc.Name AS CategoryName,
   ps.Name AS SubcategoryName,
   AVG(p.ListPrice - p.StandardCost) AS AvgProfit,
   AVG((p.ListPrice - p.StandardCost) / NULLIF(p.ListPrice, 0)) AS AvgMarginPercent,
   COUNT(DISTINCT p.ProductID) AS ProductCount
FROM General_Panel AS gp
JOIN Production.Product AS p                 ON gp.ProductID = p.ProductID
LEFT JOIN Production.ProductSubcategory AS ps  ON p.ProductSubcategoryID = ps.ProductSubcategoryID
LEFT JOIN Production.ProductCategory    AS pc  ON ps.ProductCategoryID = pc.ProductCategoryID
GROUP BY pc.Name, ps.Name
ORDER BY AvgMarginPercent DESC;

-- Product-level "loss makers" (actual)
SELECT
   ProductName,
   CategoryName,
   SUM(LineRevenue) AS TotalRevenue,
   SUM(LineCost)    AS TotalCost,
   SUM(LineProfit)  AS TotalProfitLoss
FROM General_Panel
GROUP BY
   ProductName,
   CategoryName
HAVING
   SUM(LineProfit) < 0
ORDER BY
   TotalProfitLoss ASC;   -- most negative first

-- Profit by Product Category (real sales)
SELECT
   CategoryName,
   AVG(LineProfit)    AS AvgProfit,
   SUM(LineProfit)    AS TotalProfit,
   AVG(MarginPercent) AS AvgMarginPercent,
   COUNT(*)           AS Transactions,
   SUM(OrderQty)      AS TotalSold
FROM General_Panel
GROUP BY CategoryName
ORDER BY TotalProfit DESC;

-- Profit by Product Subcategory (real sales)
SELECT
   SubcategoryName,
   CategoryName,
   AVG(LineProfit)    AS AvgProfit,
   SUM(LineProfit)    AS TotalProfit,
   AVG(MarginPercent) AS AvgMarginPercent,
   COUNT(*)           AS Transactions,
   SUM(OrderQty)      AS TotalSold
FROM General_Panel
GROUP BY SubcategoryName, CategoryName
ORDER BY TotalProfit DESC;

-- Top 100 products by total profit
SELECT TOP 100
   ProductID,
   ProductName,
   CategoryName,
   SubcategoryName,
   AVG(LineProfit)    AS AvgProfit,
   SUM(LineProfit)    AS TotalProfit,
   AVG(MarginPercent) AS AvgMarginPercent,
   COUNT(*)           AS SalesCount,
   SUM(OrderQty)      AS TotalSold
FROM General_Panel
GROUP BY
   ProductID, ProductName, CategoryName, SubcategoryName
ORDER BY TotalProfit DESC;

-- 60 least profitable products
SELECT TOP 60
   ProductID,
   ProductName,
   CategoryName,
   SubcategoryName,
   AVG(LineProfit)    AS AvgProfit,
   SUM(LineProfit)    AS TotalProfit,
   AVG(MarginPercent) AS AvgMarginPercent,
   COUNT(*)           AS SalesCount,
   SUM(OrderQty)      AS TotalSold
FROM General_Panel
GROUP BY
   ProductID, ProductName, CategoryName, SubcategoryName
ORDER BY TotalProfit ASC;

-- Category mix by year-month (used for explaining margin swings)
SELECT
   OrderYear,
   OrderMonthNumber,
   CategoryName,
   SUM(LineRevenue) AS Revenue,
   SUM(LineProfit)  AS Profit
FROM General_Panel
GROUP BY OrderYear, OrderMonthNumber, CategoryName
ORDER BY OrderYear, OrderMonthNumber, Revenue DESC;

/* ============================================================
  PHASE 5 – NEGATIVE PROFIT DIAGNOSTICS & COVERAGE
  ============================================================ */

-- Top 10 most unprofitable order lines
SELECT TOP 10
   ProductID,
   ProductName,
   OrderQty,
   UnitPrice,
   StandardCost,
   LineRevenue,
   LineProfit,
   MarginPercent
FROM General_Panel
ORDER BY LineProfit ASC; -- most negative first

-- Top 10 most profitable order lines
SELECT TOP 10
   ProductID,
   ProductName,
   OrderQty,
   UnitPrice,
   StandardCost,
   LineRevenue,
   LineProfit,
   MarginPercent
FROM General_Panel
ORDER BY LineProfit DESC;

-- Aggregate view of all negative-profit lines
SELECT
   COUNT(*)           AS NegativeCount,
   AVG(LineProfit)    AS AvgNegativeProfit,
   AVG(MarginPercent) AS AvgNegativeMargin,
   AVG(DiscountPercent) AS AvgDiscount,
   AVG(OrderQty)      AS AvgQty
FROM General_Panel
WHERE LineProfit < 0;

-- Negative-profit lines by category
SELECT
   CategoryName,
   COUNT(*)           AS NegativeCount,
   AVG(MarginPercent) AS AvgMargin
FROM General_Panel
WHERE LineProfit < 0
GROUP BY CategoryName
ORDER BY NegativeCount DESC;

-- Products that never sold vs total products
SELECT
   COUNT(DISTINCT b.ProductID) AS SoldProductCount,
   COUNT(DISTINCT a.ProductID) AS TotalProductCount
FROM Production.Product AS a
LEFT JOIN General_Panel AS b
      ON a.ProductID = b.ProductID;

/* ============================================================
  PHASE 6 – EXTRA: FOCUSED SUBCATEGORY ANALYSIS
  (example on specific subcategories – Jerseys, Caps, Frames, etc.)
  ============================================================ */

SELECT
   OrderYear,
   CategoryName,
   SubCategoryName,
   SUM(LineProfit)                 AS TotalProfitForSubCategory,
   SUM(LineProfit)/SUM(LineRevenue)   AS TotalMarginForSubCategory,
   SUM(LineRevenue)                 AS TotalRevenueForSubCategory,
   SUM(OrderQty)                   AS SumQtyOfProductsSoldInSubCategory,
   AVG(DiscountPercent)             AS AvgDiscountPerSubCategory,
   AVG(UnitPrice)                  AS AvgUnitPriceForSubCategory,
   AVG(NetUnitPrice)                AS AvgNetUnitPriceForSubCategory,
   SUM(UnitPrice)                  AS TotalUnitPriceForSubCategory,
   SUM(NetUnitPrice)                AS TotalNetUnitPriceForSubCategory
FROM General_Panel
WHERE SubCategoryName IN ('Jerseys', 'Caps', 'Road Frames', 'Touring Bikes', 'Touring Frames')
GROUP BY SubCategoryName, CategoryName, OrderYear
ORDER BY OrderYear;

/* ============================================
  PRODUCTS WITH NEGATIVE LINES – ROOT CAUSE
  Are they inherently unprofitable, or sold too cheap?
  ============================================ */

WITH NegativeLines AS (
   SELECT DISTINCT ProductID
   FROM General_Panel
   WHERE LineProfit < 0
),

ProductSummary AS (
   SELECT
      gp.ProductID,
      gp.ProductName,
      gp.CategoryName,
      gp.SubcategoryName,

      -- Theoretical info from Product table
      p.StandardCost,
      p.ListPrice,
      (p.ListPrice - p.StandardCost) AS TheoreticalProfit,
      (p.ListPrice - p.StandardCost) / NULLIF(p.ListPrice, 0) AS TheoreticalMarginPercent,

      -- What actually happened in sales
      MIN(gp.UnitPrice) AS MinUnitPrice_Sold,
      MAX(gp.UnitPrice) AS MaxUnitPrice_Sold,

      -- How often this product is negative vs total
      SUM(CASE WHEN gp.LineProfit < 0 THEN 1 ELSE 0 END) AS NegativeLineCount,
      COUNT(*) AS TotalLineCount,

      -- Useful flags
      CASE
         WHEN p.ListPrice <= p.StandardCost THEN 1 ELSE 0
      END AS IsInherentlyUnprofitable,  -- product price < cost in master data

      MIN(gp.UnitPrice - p.StandardCost) AS MinUnitPriceMinusCost, -- if < 0 → sold below cost at least once
      MIN(gp.MarginPercent) AS MinObservedMarginPercent,
      MAX(gp.MarginPercent) AS MaxObservedMarginPercent

   FROM General_Panel AS gp
   JOIN NegativeLines AS nl
       ON gp.ProductID = nl.ProductID
   JOIN Production.Product AS p
       ON gp.ProductID = p.ProductID
   GROUP BY
      gp.ProductID,
      gp.ProductName,
      gp.CategoryName,
      gp.SubcategoryName,
      p.StandardCost,
      p.ListPrice
)

SELECT
   ProductID,
   ProductName,
   CategoryName,
   SubcategoryName,
   StandardCost,
   ListPrice,
   TheoreticalProfit,
   TheoreticalMarginPercent,
   MinUnitPrice_Sold,
   MaxUnitPrice_Sold,
   NegativeLineCount,
   TotalLineCount,
   IsInherentlyUnprofitable,
   MinUnitPriceMinusCost,
   MinObservedMarginPercent,
   MaxObservedMarginPercent
FROM ProductSummary
ORDER BY
   IsInherentlyUnprofitable DESC,   -- first: structurally bad products
   NegativeLineCount DESC;         -- then: most problematic by volume
```